多人联机、空间定位。

马思远：网络部署、软件系统安装、货架入场

李国川：播种墙、货架、划线贴码、

孙宇超：设备入场、机器人入场、充电安装

白鹏飞：流程测试、机器人联动、导入地图

Unity3D 开发规范与系统设计文档

文档目的：为团队制定统一的开发规范、模块设计原则和协作流程，降低后续手动合并的复杂度，提升代码质量和可扩展性。

适用范围：本项目所有开发人员及所有业务模块。

目标人群：初级 Unity 开发人员，规则设计以简单、可落地、易检查为核心。

一、项目整体架构设计

采用 **“核心层 + 业务模块层”** 的分层架构，核心层提供通用功能和全局管理，业务模块层独立实现具体业务，模块间通过核心层的通信机制交互，避免直接耦合。

二、统一目录结构规范

为避免文件路径混乱导致的合并冲突，所有模块必须遵循统一的目录结构，**目录命名全部使用英文，采用“大驼峰 + 下划线”（如 Role_Module）**，禁止使用中文、特殊字符或拼音混写。

三、项目根目录结构

```
├─ Modules                    // 业务模块层（各开发人员负责对应模块）
│   ├─ [ModuleName]           // 业务模块目录（如 Role_Module、Battle_Module）
│   │   ├─ Scripts            // 模块脚本（按 MVC 再细分）
│   │   │   ├─ Model          // 数据模型层
│   │   │   ├─ View           // 视图层（与 UI/场景物体交互）
│   │   │   └─ Controller     // 控制层（业务逻辑处理）
│   ├─ Prefabs                // 模块预制体（如角色预制体、道具预制体）
```

（1）模块内目录规范

1. 每个业务模块必须包含 Scripts（按 MVC 细分）、Prefabs、Resources 文件夹，无内容则保留空文件夹。
2. 脚本文件必须放在对应 MVC 子目录下，禁止跨目录存放（如 Controller 脚本不得放在 Model 目录）。
3. 资源文件按类型再细分（如 Resources/Textures、Resources/Audio），禁止杂乱存放。

四、业务模块系统设计规范

所有业务模块需遵循 **MVC 设计模式**开发，同时结合核心层的全局系统实现功能，确保模块内部结构统一，便于主程审查和合并。

层级	职责描述	开发注意事项
Model	负责模块数据的存储、读取和更新，不包含任何视图逻辑和业务逻辑	仅提供数据访问接口，禁止引用 MonoBehaviour
View	负责模块的 UI / 场景物体展示，接收用户输入并传递给 Controller，不包含业务逻辑	仅与 Controller 交互，禁止直接修改 Model 数据
Controller	负责模块的业务逻辑处理，接收 View 的输入，操作 Model 更新数据，通过事件中心通知 View 刷新	作为模块的核心入口，禁止直接操作 UI / 场景物体

五、代码注释

1. **代码注释规范**:

```
/// <summary>
/// 角色控制层：负责角色的移动、攻击、属性更新等业务逻辑
/// 作者：XXX
/// </summary>
public class RoleController : BaseController
{

}
```

- 类注释：必须包含类的职责、作者、创建时间（可选）。
- 公共方法注释：必须包含方法功能、参数说明、返回值说明。
- 复杂逻辑注释：关键业务逻辑行必须添加行注释，说明设计思路

六、硬性规定

1、每个模块增加未完成状态和已完成状态

2、每个模块可以随意移动位置

3、尽量使用不拖拽处理执行处理，能在代码上写的就在代码上写。